

RAPPORT DE TEST D'INTRUSION

Site web de Bug&Blog Ltd

client

Bug&Blog Ltd

Date

2025-02-28

Rédigé par

Lisa-Marie Ciszak

SUMMARY

1. Introduction.....	3
2. Porté du test.....	4
3. Synthèse de la mission.....	5
4. Résumé.....	6
4.1 Surface vulnérable.....	6
4.2 Table des vulnérabilités.....	6
5. Vulnérabilités.....	7
5.2 Anonymous Diffie-Hellman Key Exchange MitM.....	7
5.3 Cross-Site Request Forgery (CSRF).....	10
5.4 Accès Anonyme autorisé au serveur FTP.....	15
5.5 Exposition d'informations sensibles via SNMP.....	19
5.6 Accès fichier sensible sans avoir de droit (robot.txt, config.php).....	23
5.7 Vulnérabilité de Gestion des Mots de Passe - WordPress et Base de Données.....	26
6. Conclusion.....	29

1. Introduction

Ce document présente les résultats d'un test d'intrusion effectué sur le site web de **Bug&Blog Ltd.** L'objectif de cette évaluation de sécurité était d'identifier les vulnérabilités qui pourraient compromettre la sécurité du site, affecter les données qu'il traite et, par extension, nuire à l'entreprise.

Afin de tester la résilience du site face à des scénarios d'attaque réels, le prestataire a simulé des attaques de manière méthodique. Ce processus a permis de mettre en lumière les failles potentielles dans la sécurité du site, qu'elles soient liées à des erreurs de configuration ou à des vulnérabilités spécifiques aux applications web. En analysant ces risques, cette mission vise à fournir une évaluation complète de l'impact possible de chaque vulnérabilité et à proposer des actions correctives pour améliorer la sécurité du site.

2. Porté du test

La portée du test d'intrusion était limitée au site web Bug&Blog Ltd et le serveur hébergeant la solution.

L'évaluation a porté sur les points suivants:

Host
192.31.25.12

3. Synthèse de la mission

La mission a été réalisée dans une période de 4 heures. Le test d'intrusion a commencé le 28 février 2025 et s'est terminé le jour même avec la remise de la version finale de ce rapport.

Toutes les activités de test se sont déroulées dans l'environnement mis à disposition. Le site web a été analysé à l'aide d'outils mais a principalement fait l'objet de tests manuels.

Les attaques ont été menées depuis ma machine qui a pour adresse 192.31.25.13.

4. Résumé

Bug&Blog Ltd a sollicité les services d'une experte en cybersécurité pour réaliser un test d'intrusion de leur site web. Cette plateforme permet aux utilisateurs de partager des informations liées à leurs projets.

Le test d'intrusion avait pour objectif principal d'identifier et d'exploiter un maximum de vulnérabilités, afin d'évaluer le niveau de sécurité du site face à des attaques réelles. La méthodologie appliquée s'est concentrée sur l'identification des failles de sécurité susceptibles de conduire à la divulgation d'informations sensibles et sur les accès non autorisés pouvant compromettre la plateforme ou ses utilisateurs.

4.1 Surface vulnérable

Le site web présentait un niveau de sécurité jugé insuffisant compte tenu du nombre de vulnérabilités découvertes, de leur sévérité et du risque que ces problèmes pourraient représenter pour le système et les données qu'il traite.

4.2 Table des vulnérabilités

Le tableau suivant résume les vulnérabilités trouvées dans l'application.

Vulnerability	Severity
Anonymous Diffie-Hellman Key Exchange MitM	Critical
Cross-Site Request Forgery (CSRF)	Medium
Accès Anonyme autorisé au serveur FTP	High
Exposition d'informations sensibles via SNMP	High
Accès fichier sensible sans avoir de droit (robot.txt, config.php)	High
Vulnérabilité de Gestion des Mots de Passe - WordPress et Base de Données	Critical

5. Vulnérabilités

5.2 Anonymous Diffie-Hellman Key Exchange MitM

Severity	CWE ID	CVSS Score
Critical	CWE-327	9.1

```
PORT STATE SERVICE
25/tcp open  smtp
| ssl-dh-params:
| VULNERABLE:
| Anonymous Diffie-Hellman Key Exchange MitM Vulnerability
| State: VULNERABLE
| Transport Layer Security (TLS) services that use anonymous
| Diffie-Hellman key exchange only provide protection against passive
| eavesdropping, and are vulnerable to active man-in-the-middle attacks
| which could completely compromise the confidentiality and integrity
| of any data exchanged over the resulting session.
```

Description

Le serveur SMTP cible utilise l'échange de clés Diffie-Hellman anonyme (DH-ANON) dans sa configuration TLS.

Ce mode d'échange ne repose pas sur des certificats d'authentification, ce qui signifie qu'un attaquant interceptant la communication peut facilement se placer en position d'homme du milieu (Man-in-the-Middle - MitM) et déchiffrer ou modifier les données échangées entre le client et le serveur.

L'absence d'authentification dans cet échange de clés empêche de garantir que le serveur avec lequel le client communique est bien celui qu'il prétend être.

Cette vulnérabilité est référencée dans la norme TLS (RFC 2246), qui précise que les suites de chiffrement DH-ANON ne doivent pas être utilisées en raison de leur manque de sécurité.

Impact

Un attaquant pourrait exploiter cette faiblesse pour :

- **Intercepter et déchiffrer les communications SMTP sécurisées** (ex : mails envoyés avec STARTTLS).
- **Altérer les messages en transit**, injecter des instructions malveillantes ou modifier le contenu des emails.
- **Usurper l'identité du serveur SMTP**, ce qui pourrait conduire à l'envoi de faux e-mails ou au vol de données confidentielles.

Dans un contexte professionnel, cette faille pourrait être utilisée pour :

- **Voler des identifiants et mots de passe SMTP.**
- **Récupérer des emails sensibles** contenant des informations confidentielles.
- **Servir de point d'entrée pour des attaques plus avancées** (ex : spear phishing, attaque sur d'autres services internes).

Reproduction

Scénario d'attaque : interception et déchiffrement de mails

L'attaquant se positionne en homme du milieu (MitM) en redirigeant le trafic SMTP (ex : via ARP spoofing ou détournement DNS).

Il intercepte une connexion STARTTLS entre un client SMTP et le serveur ciblé sur le port 25/tcp.

Comme l'échange de clés DH-ANON ne fournit aucune authentification, l'attaquant peut établir deux connexions distinctes :

- Une avec le client (en se faisant passer pour le serveur).
- Une autre avec le serveur (en se faisant passer pour le client).

Il intercepte et déchiffre toutes les communications SMTP chiffrées via STARTTLS.

Il peut ainsi récupérer des identifiants SMTP, des mails envoyés, ou modifier les messages en transit.

`nmap (nmap --script=ssl-dh-params -p 25 192.31.25.12).`

Recommandations

- **Désactiver l'échange de clés Diffie-Hellman anonyme (DH-ANON)**

Modifier la configuration du serveur SMTP pour exclure les suites de chiffrement utilisant TLS_DH_anon.

- **Forcer l'utilisation de chiffrement sécurisé**

Autoriser uniquement des chiffrements forts avec authentification, comme :

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384

- **Utiliser des certificats valides et signés**

Vérifier que le serveur SMTP utilise un certificat TLS valide et signé par une autorité de certification reconnue.

Éviter les certificats auto-signés sauf si un PKI interne est en place.

- **Activer Perfect Forward Secrecy (PFS)**

S'assurer que le serveur supporte ECDHE pour éviter que des communications passées ne soient déchiffrées en cas de compromission de la clé privée.

- **Vérifier la configuration avec un scanner de vulnérabilité**

Conclusion

La présence d'une suite de chiffrement DH-ANON sur le serveur SMTP représente un risque majeur pour la sécurité des communications. Elle permet à un attaquant de réaliser une attaque Man-in-the-Middle, d'intercepter et de manipuler les emails envoyés via STARTTLS.

L'exclusion de ces suites de chiffrement, couplée à une configuration stricte de TLS, est nécessaire avant toute mise en production du serveur SMTP

5.3 Cross-Site Request Forgery (CSRF)

Severity	CWE ID	CVSS Score
Medium	CWE-352	4.3

```

PORT    STATE SERVICE
80/tcp  open  http
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-enum:
|   /wp-login.php: Possible admin folder
|   /wp-json: Possible admin folder
|   /robots.txt: Robots file
|   /readme.html: Wordpress version: 2
|   /: Wordpress version: 4.6.29
|   /feed/: Wordpress version: 4.6.29
|   /wp-includes/images/rss.png: Wordpress version 2.2 found.
|   /wp-includes/js/jquery/suggest.js: Wordpress version 2.5 found.
|   /wp-includes/images/blank.gif: Wordpress version 2.6 found.
|   /wp-includes/js/comment-reply.js: Wordpress version 2.7 found.
|   /wp-login.php: Wordpress login page.
|   /wp-admin/upgrade.php: Wordpress login page.
|   /readme.html: Interesting, a readme.
|   /0/: Potentially interesting folder
|   /contact/: Potentially interesting folder
|   /private/: Potentially interesting folder
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.

```

```

PORT    STATE SERVICE
80/tcp  open  http
|_http-wordpress-users:
|   Username found: warbox
|_Search stopped at ID #25. Increase the upper limit if necessary with 'http-wordpress-users.limit'
|_http-csrf:
|   Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=192.31.25.12
|   Found the following possible CSRF vulnerabilities:

```

Description

Le site WordPress hébergé sur le serveur cible est vulnérable à une attaque de type Cross-Site Request Forgery (CSRF).

Cette vulnérabilité permet à un attaquant d'exécuter des actions à l'insu d'un utilisateur authentifié, sans son consentement, en exploitant sa session active sur le site.

Dans une attaque CSRF, une victime authentifiée sur le site est incitée à cliquer sur un lien malveillant ou à charger une page contenant une requête cachée. Cette requête est alors exécutée avec les privilèges de la victime, permettant à l'attaquant d'effectuer des actions telles que :

- **Modifier des paramètres du site** (ex : changement d'email ou de mot de passe).
- **Publier du contenu malveillant sur le blog.**
- **Créer un nouvel utilisateur administrateur.**
- **Modifier ou supprimer des articles.**

Le test Nmap a révélé plusieurs formulaires potentiellement vulnérables sur différentes pages du site, notamment la page de connexion (**/wp-login.php**).

Impact

L'exploitation d'une attaque CSRF pourrait avoir des conséquences graves pour le site WordPress et l'entreprise Bug&Blog Ltd :

- Prise de contrôle du compte d'un utilisateur authentifié, y compris un administrateur.
- Modification des paramètres WordPress pour rediriger les utilisateurs vers un site malveillant.
- Injection de contenu malveillant pouvant mener à du phishing ou à la diffusion de logiciels malveillants.
- Suppression de contenus importants, impactant l'intégrité du blog.
- Atteinte à la réputation de Bug&Blog Ltd en cas de compromission publique du site.

Reproduction

Scénario d'attaque : modification du mot de passe administrateur

L'attaquant crée une page web malveillante contenant un formulaire caché qui envoie une requête POST vers `http://192.31.25.12/wp-admin/profile.php` pour modifier le mot de passe.

Exemple de code HTML injecté :

```
<form action="http://192.31.25.12/wp-admin/profile.php" method="POST">
```

```
  <input type="hidden" name="pass1" value="MotDePassePirate123">
```

```
  <input type="hidden" name="pass2" value="MotDePassePirate123">
```

```
  <input type="hidden" name="submit" value="Enregistrer">
```

```
</form>
```

```
<script>
```

```
  document.forms[0].submit();
```

```
</script>
```

Il incite un administrateur connecté à visiter cette page (ex : via un email de phishing ou une redirection sur un site tiers). Dès que la page est chargée, le formulaire s'exécute automatiquement et le mot de passe de l'administrateur est modifié. L'attaquant peut alors se connecter avec le nouveau mot de passe et prendre le contrôle total du site.

Outils permettant de tester l'attaque :

- Burp Suite : pour capturer et modifier les requêtes HTTP.
- Nmap avec `--script=http-csrf` : pour scanner les pages vulnérables.

Recommandations

- **Implémenter un jeton CSRF (CSRF Token) pour chaque requête sensible**

Ajouter un jeton unique à chaque formulaire sensible (ex : modification de mot de passe, publication d'articles).

Exemple d'implémentation en PHP :

```
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

Dans le formulaire :

```
<input type="hidden" name="csrf_token" value="php echo<br/$_SESSION['csrf_token']; ?>">
```

Et lors du traitement :

```
if ($_POST['csrf_token'] !== $_SESSION['csrf_token']) {  
  
    die("CSRF détecté !");  
  
}
```

- **Forcer l'utilisation de requêtes POST protégées**

Vérifier que les requêtes modifiant des données ne peuvent pas être envoyées via une simple URL (GET).

Désactiver toute action critique en GET, comme :

<http://192.31.25.12/delete-post?id=5>

- **Vérifier l'origine des requêtes HTTP (SameSite & Referer)**

Activer la directive SameSite pour les cookies :

Set-Cookie: session_id=abc123; Secure; HttpOnly; SameSite=Strict

Vérifier l'en-tête Referer avant d'exécuter des actions critiques.

- **Restreindre l'accès aux pages sensibles**

Exiger une **re-authentification** avant toute modification critique (changement d'email, modification du mot de passe et création ou suppression d'un utilisateur)

- **Utiliser un plugin de sécurité WordPress**

Installer un plugin de protection CSRF comme Wordfence Security ou All In One WP Security & Firewall.

Conclusion

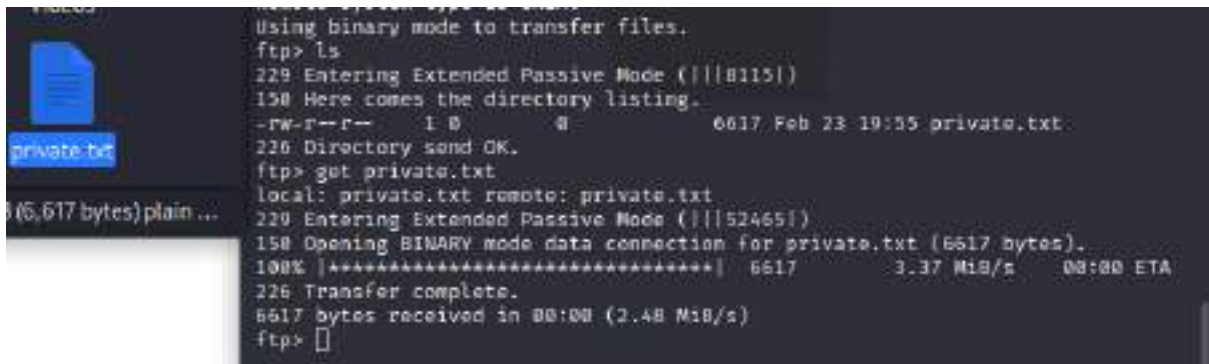
La vulnérabilité CSRF détectée sur le serveur WordPress de Bug&Blog Ltd permet à un attaquant d'exécuter des actions à l'insu des utilisateurs authentifiés. Un attaquant pourrait prendre le contrôle du site en modifiant des paramètres sensibles ou en injectant du contenu malveillant.

L'implémentation de jetons CSRF, la restriction des requêtes GET sur les actions sensibles et l'ajout d'un contrôle des origines des requêtes sont essentiels avant la mise en ligne du site.

5.4 Accès Anonyme autorisé au serveur FTP

Severity	CWE ID	CVSS Score
High	CWE-287	7.5

```
$ nmap -script ftp-anon 192.31.25.12
Starting Nmap 7.93 ( https://nmap.org ) at 2025-02-28 08:07 EST
Nmap scan report for 192.31.25.12
Host is up (0.00057s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_-rw-r--r--  1 0      0      6617 Feb 23 19:55 private.txt
```



```
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||B115|)
150 Here comes the directory listing.
-rw-r--r--  1 0      0      6617 Feb 23 19:55 private.txt
226 Directory send OK.
ftp> get private.txt
local: private.txt remote: private.txt
229 Entering Extended Passive Mode (|||52465|)
150 Opening BINARY mode data connection for private.txt (6617 bytes).
100% |*****| 6617 3.37 MiB/s 00:00 ETA
226 Transfer complete.
6617 bytes received in 00:00 (2.48 MiB/s)
ftp>
```

Description

Le serveur FTP sur le port 21 autorise les connexions anonymes, permettant à quiconque de se connecter sans authentification en utilisant l'identifiant anonymous. Lors de l'analyse, il a été constaté qu'un fichier private.txt est accessible en lecture pour tout utilisateur anonyme.

Un accès FTP anonyme mal sécurisé peut entraîner une fuite d'informations sensibles, telles que des fichiers de configuration, des identifiants stockés en clair ou des logs contenant des informations sur l'infrastructure du serveur.

Impact

Un attaquant pourrait télécharger des fichiers sensibles hébergés sur le serveur FTP ainsi qu'exploiter des informations contenues dans ces fichiers pour mener des attaques ciblées (ex. vol de mots de passe, reconnaissance du système). Il pourrait également utiliser le FTP comme point d'entrée pour injecter du contenu malveillant ou préparer une future attaque sur d'autres services du serveur.

Si le serveur FTP permet également l'écriture de fichiers, l'attaquant pourrait téléverser des fichiers malveillants, facilitant ainsi une prise de contrôle du serveur.

Reproduction

Un attaquant peut exploiter cette vulnérabilité en suivant ces étapes :

Se connecter anonymement au serveur FTP

ftp 192.31.25.12

Lorsque l'invite demande un identifiant, entrer :

Name: anonymous

Lister les fichiers accessibles

ftp> ls

Résultat observé :

```
-rw-r--r--  1 0      0      6617 Feb 23 19:55 private.txt
```

Télécharger le fichier private.txt

ftp> get private.txt

Une fois téléchargé, son contenu peut être affiché avec :

cat private.txt

Si ce fichier contient des données sensibles (ex. mots de passe, configurations réseau, logs), l'attaquant peut les exploiter pour compromettre d'autres services.

Recommandations

- Désactiver l'accès anonyme au FTP

Modifier la configuration du serveur FTP (vsftpd.conf, proftpd.conf, etc.).

Pour vsftpd, ouvrir /etc/vsftpd.conf et s'assurer que la ligne suivante est présente :

```
anonymous_enable=NO
```

Redémarrer le service :

```
sudo systemctl restart vsftpd
```

- Restreindre l'accès aux fichiers sensibles

Modifier les permissions des fichiers accessibles par le FTP :

```
chmod 600 /chemin/vers/private.txt
```

Cela empêche les utilisateurs anonymes et non autorisés d'y accéder.

- Activer une authentification forte

Exiger une connexion avec un nom d'utilisateur et un mot de passe sécurisé et utiliser SFTP (FTP sécurisé via SSH) à la place du FTP classique.

- Fermer le port 21 si FTP n'est pas nécessaire

Si le FTP n'est pas indispensable, mieux vaut désactiver le service :

```
sudo systemctl stop vsftpd
```

```
sudo systemctl disable vsftpd
```

Alternativement, restreindre l'accès au FTP avec un pare-feu (ex. UFW sous Ubuntu) :

```
sudo ufw deny 21/tcp
```

Conclusion

L'accès FTP anonyme représente une faille de sécurité majeure car il permet à n'importe qui de lister et télécharger des fichiers potentiellement sensibles. Il est impératif de désactiver cet accès, de restreindre les permissions des fichiers et d'implémenter une authentification forte avant la mise en production du serveur.

5.5 Exposition d'informations sensibles via SNMP

Severity	CWE ID	CVSS Score
High	CWE-200	7.5

```
└─$ snmp-check 192.31.25.12
snmp-check v1.9 - SNMP enumerator
Copyright (c) 2005-2015 by Matteo Cantoni (www.nothink.org)

[+] Try to connect to 192.31.25.12:161 using SNMPv1 and community 'public'

[*] System information:

  Host IP address      : 192.31.25.12
  Hostname             : warbox-01
  Description          : Linux warbox-01 4.15.0-213-generic #224-Ubu
ntu SMP Mon Jun 19 13:30:12 UTC 2023 x86_64
  Contact              : Rick <rickastley@iloveit.com>
  Location             : __WARBOX-FLAG__
  Uptime snmp          : 00:34:00.78
  Uptime system        : 00:33:51.47
  System date          : 2025-2-28 13:18:56.0

[*] Network information:

  IP forwarding enabled : no
  Default TTL           : 64
  TCP segments received : 18845
```

Description

Le service SNMP (Simple Network Management Protocol) est activé sur l'hôte 192.31.25.12 et permet l'accès en lecture aux informations système via la communauté public.

Cela signifie qu'un attaquant peut interroger ce service sans authentification et récupérer des informations sensibles sur le système et le réseau.

Impact

Un attaquant exploitant cette vulnérabilité pourrait récupérer une série d'informations système sensibles, telles que le nom de l'hôte, la version du noyau et du système d'exploitation, ainsi que le temps de fonctionnement du système. Ces données peuvent fournir des pistes cruciales pour déterminer les points d'entrée potentiels et les versions vulnérables à exploiter.

De plus, il pourrait obtenir des informations réseau critiques, y compris l'adresse IP de l'hôte, les interfaces réseau et leurs configurations, ainsi que des statistiques d'utilisation du trafic réseau. Ces informations permettent une reconnaissance détaillée du réseau et la possibilité de localiser d'autres machines vulnérables.

L'attaquant pourrait aussi collecter des informations sur les utilisateurs et services en cours d'exécution, telles que les comptes administrateurs et utilisateurs connectés, ainsi que les services et ports ouverts. Il pourrait également obtenir des données sur l'emplacement et les contacts renseignés, des informations qui peuvent être utilisées pour des attaques d'ingénierie sociale.

En exploitant ces informations, l'attaquant pourrait planifier des attaques ciblées, comme l'exploitation de vulnérabilités du noyau Linux. Il serait aussi en mesure de mener une reconnaissance avancée du réseau pour identifier d'autres machines vulnérables, et effectuer des attaques de type spoofing ou Man-in-the-Middle (MitM) en compromettant des équipements réseau, augmentant ainsi l'ampleur de l'attaque et son impact.

Reproduction

L'attaque est reproductible en utilisant un simple scan SNMP:

```
snmp-check 192.31.25.12
```

Cela prouve que SNMP est accessible en lecture sans authentification, permettant l'exfiltration de données sensibles.

Recommandations

- **Désactiver SNMP si non utilisé**

Si le protocole SNMP n'est pas indispensable, il est recommandé de le désactiver :

```
sudo systemctl stop snmpd
```

```
sudo systemctl disable snmpd
```

- **Restreindre l'accès SNMP aux adresses IP autorisées**

Dans /etc/snmp/snmpd.conf, limiter l'accès à certaines IP de confiance :

```
rocommunity MyS3cur3C0mmun1ty 192.168.1.0/24
```

- **Désactiver SNMPv1 et SNMPv2, utiliser SNMPv3**

SNMPv1 et SNMPv2 ne chiffrent pas les données et SNMPv3 permet l'authentification et le chiffrement

- **Modifier la communauté SNMP par défaut**

La communauté public est une valeur par défaut bien connue. Elle doit être changée dans /etc/snmp/snmpd.conf :

```
rocommunity MyS3cur3C0mmun1ty
```

Puis redémarrer le service :

```
sudo systemctl restart snmpd
```

- **Configurer un pare-feu pour bloquer SNMP en dehors du réseau interne**

Bloquer l'accès au port **161/UDP** depuis l'extérieur avec ufw :

```
sudo ufw deny from any to any port 161 proto udp
```

Conclusion

L'exposition du service SNMP avec la communauté par défaut public représente une faille critique, permettant la fuite d'informations système et réseau.

Une correction rapide est nécessaire en désactivant SNMP ou en sécurisant son accès avec SNMPv3 et des restrictions d'IP.

5.6 Accès fichier sensible sans avoir de droit (robot.txt, config.php)

Severity	CWE ID	CVSS Score
High	CWE-552	7.5

```
define('DB_NAME', 'wordpress');  
  
/** MySQL database username */  
define('DB_USER', 'wordpress');  
  
/** MySQL database password */  
define('DB_PASSWORD', 'ProGTR00SQL');  
  
/** MySQL hostname */  
define('DB_HOST', 'localhost');
```

Description

L'accès aux fichiers sensibles sur un serveur Web, comme le fichier wp-config.php de WordPress et robots.txt, expose des informations critiques pour la sécurité du système.

Le fichier wp-config.php contient des informations de connexion à la base de données MySQL, telles que le nom d'utilisateur, le mot de passe et le nom de la base de données. Ces informations permettent à un attaquant de se connecter directement à la base de données et d'exécuter des requêtes, potentiellement pour exécuter du code arbitraire, voler des informations ou perturber le bon fonctionnement du site Web.

Le fichier robots.txt fournit des informations sur les répertoires ou fichiers que le serveur Web préfère que les moteurs de recherche n'indexent pas. Bien que ce fichier ne contienne pas directement des informations sensibles, il peut indiquer des fichiers ou répertoires qui pourraient être utilisés à des fins malveillantes si l'attaquant est déjà sur le serveur.

Impacte

Un attaquant pourrait exploiter ces informations pour :

- Se connecter à la base de données MySQL et manipuler les données.
- Effectuer des attaques par injection SQL pour accéder à l'administration WordPress, modifier des utilisateurs ou des informations sensibles.
- Accéder à des répertoires et fichiers non protégés qui pourraient contenir des informations sensibles.
- Bypasser des contrôles de sécurité en obtenant des informations privilégiées sur la configuration du serveur ou du site Web.

Reproduction

En accédant au serveur en tant qu'utilisateur SSH, il est possible de lire le contenu du fichier wp-config.php dans le répertoire d'installation de WordPress, ce qui permet d'extraire les informations suivantes :

- Nom de la base de données (DB_NAME)
- Utilisateur de la base de données (DB_USER)
- Mot de passe de la base de données (DB_PASSWORD)

Ces informations permettent ensuite de se connecter à la base de données MySQL en utilisant la commande suivante :

```
mysql -u DB_USER -p
```

Le mot de passe trouvé dans wp-config.php permet à un attaquant de se connecter à la base de données MySQL. De plus, l'exécution de requêtes SQL peut exposer des informations sensibles stockées dans les tables, comme des noms d'utilisateur et des mots de passe de WordPress, ce qui peut entraîner un accès non autorisé à l'administration du site Web.

Recommandations

- **Protéger les fichiers sensibles :**

Assurez-vous que des fichiers comme wp-config.php sont correctement protégés avec des permissions d'accès restrictives. Idéalement, ces fichiers ne devraient être accessibles qu'à l'utilisateur Web et aux administrateurs du serveur.

Commande pour restreindre l'accès au fichier wp-config.php :

```
chmod 600 wp-config.php
```

- **Utiliser un mot de passe fort pour MySQL :**

Les mots de passe trouvés dans wp-config.php doivent être forts et uniques. Envisagez d'utiliser des mots de passe générés aléatoirement ou des gestionnaires de mots de passe pour augmenter la sécurité.

- **Désactiver l'accès SSH aux utilisateurs non autorisés :**

Limiter les utilisateurs qui peuvent se connecter via SSH, en particulier aux utilisateurs ayant des privilèges limités. Utilisez des clés SSH pour l'authentification au lieu de mots de passe.

- **Protéger les répertoires sensibles :**

Ne laissez pas des répertoires comme /wp-admin/ ou /wp-includes/ accessibles sans restrictions adéquates. Utilisez des fichiers .htaccess ou des règles de pare-feu pour restreindre l'accès.

- **Surveiller les fichiers sensibles :**

Mettez en place des alertes pour toute modification de fichiers sensibles comme wp-config.php ou d'autres fichiers de configuration de WordPress.

5.7 Vulnérabilité de Gestion des Mots de Passe - WordPress et Base de Données

Severity	CWE ID	CVSS Score
Critical	CWE-255	9.1

```
mysql> SELECT * FROM wp_users;
+----+-----+-----+-----+-----+-----+-----+-----+
| ID | user_login | user_pass | user_activation_key | user_status | display_name | user_nicename | user_email | user_url | user_registered |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | warbox | $P$Km2Hf7q88LSm1f9RMghn0r4/tpl0sk/ |  | 0 | warbox | warbox | root@warbox.lhost |  | 2025-02-21 11:11 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Description :

Lors de l'audit de sécurité de la plateforme WordPress et de la base de données associée, il a été constaté que les mots de passe utilisés pour l'utilisateur WordPress et la base de données MySQL étaient insuffisamment complexes et pouvaient facilement être devinés. Ces mots de passe faibles présentent un risque important en matière de sécurité.

J'ai deviné le mot de passe de WordPress grâce au compte warbox fournit par Bug&Blog Ltd qui était ProGTR00, en voyant le mot de passe de MySQL, j'en ai déduit que le mot de passe WordPress était donc similaire.

Impact :

L'utilisation de mots de passe simples et prévisibles pour l'utilisateur WordPress et la base de données expose la machine à plusieurs risques :

Des attaquants pourraient facilement deviner les mots de passe grâce à des techniques telles que l'attaque par dictionnaire ou brute force, donnant accès aux systèmes sensibles.

Un attaquant ayant accès à un des systèmes (par exemple WordPress ou MySQL) pourrait facilement enchaîner pour accéder à d'autres ressources internes, voire obtenir des privilèges d'administration sur le serveur.

Les mots de passe similaires utilisés sur plusieurs couches du système (WordPress, base de données, compte utilisateur système) permettent à une seule compromission d'affecter plusieurs systèmes simultanément.

Reproduction :

Accès aux mots de passe simples :

Le mot de passe de l'utilisateur warbox est ProGTR00WP. Le mot de passe de la base de données MySQL est ProGTR00SQL. Le mot de passe de l'utilisateur warbox fourni par l'entreprise est ProGTR00 et on remarque également que ROOT est le même que warbox.

Deviner les mots de passe :

La structure des mots de passe est similaire et relativement simple, ce qui permet à un attaquant de deviner facilement les différents mots de passe en utilisant des techniques d'attaque telles que le brute force ou le dictionnaire.

Recommandations :

- Renforcement des mots de passe

Il est impératif de définir des mots de passe plus complexes pour tous les comptes utilisateurs, la base de données et le système en général. Ceux-ci doivent être longs (au moins 12-16 caractères) et comporter une combinaison de lettres (majuscules et minuscules), de chiffres et de caractères spéciaux.

- Utilisation d'un gestionnaire de mots de passe :

Utiliser un gestionnaire de mots de passe pour générer et stocker des mots de passe uniques pour chaque service. Cela permet de réduire les risques associés à la réutilisation de mots de passe.

- Authentification à deux facteurs (2FA) :

Mettre en place une authentification à deux facteurs pour les accès administratifs critiques, en particulier pour WordPress et la base de données, afin d'ajouter une couche de sécurité supplémentaire.

- Audit régulier des mots de passe et des configurations de sécurité

Effectuer des audits réguliers pour s'assurer que les mots de passe restent sécurisés et conformes aux bonnes pratiques de gestion des mots de passe. L'audit doit inclure la vérification des politiques de mot de passe et des configurations de sécurité sur toutes les couches du système.

Conclusion :

La gestion des mots de passe dans le système est une vulnérabilité importante. Les mots de passe simples et prévisibles rendent le système facile à attaquer. Il est essentiel de renforcer la sécurité en appliquant des pratiques modernes de gestion des mots de passe et en prenant des mesures pour empêcher les accès non autorisés.

6. Conclusion

L'objectif principal de cette analyse de sécurité était d'identifier les vulnérabilités existantes sur le système cible, afin d'aider l'organisation à évaluer les risques potentiels et à renforcer sa posture de sécurité face aux menaces. L'audit a révélé plusieurs failles importantes qui, si elles sont exploitées par des attaquants, pourraient avoir des conséquences graves pour la confidentialité, l'intégrité et la disponibilité des données, ainsi que pour l'ensemble de l'infrastructure.

Parmi les vulnérabilités identifiées, on note des problèmes de configuration tels que l'accès anonyme au serveur FTP, la gestion inadéquate des mots de passe sur WordPress et dans la base de données, ainsi que l'exposition d'informations sensibles via des fichiers non protégés comme le fichier config.php. De plus, des failles web comme le Cross-Site Request Forgery (CSRF) et l'usage d'un échange de clés Diffie-Hellman anonyme (vulnérable aux attaques Man-in-the-Middle) accentuent le risque de compromission.

Ces vulnérabilités représentent un danger réel pour l'organisation, notamment en permettant à des attaquants d'accéder à des informations sensibles, de manipuler des données, ou de compromettre l'intégrité du réseau. Si ces failles ne sont pas corrigées rapidement, elles pourraient être exploitées pour mener des attaques ciblées, compromettant ainsi l'ensemble de l'écosystème informatique.

Il est donc impératif que des mesures de sécurité appropriées soient mises en place pour corriger ces vulnérabilités et limiter les risques associés. Cela inclut la mise à jour des configurations, l'application de bonnes pratiques de gestion des mots de passe, la sécurisation des accès au serveur, ainsi que l'audit régulier des services exposés. Il est également recommandé de former les administrateurs à une gestion proactive des risques de sécurité et de réaliser une surveillance continue pour détecter et prévenir toute tentative d'attaque.